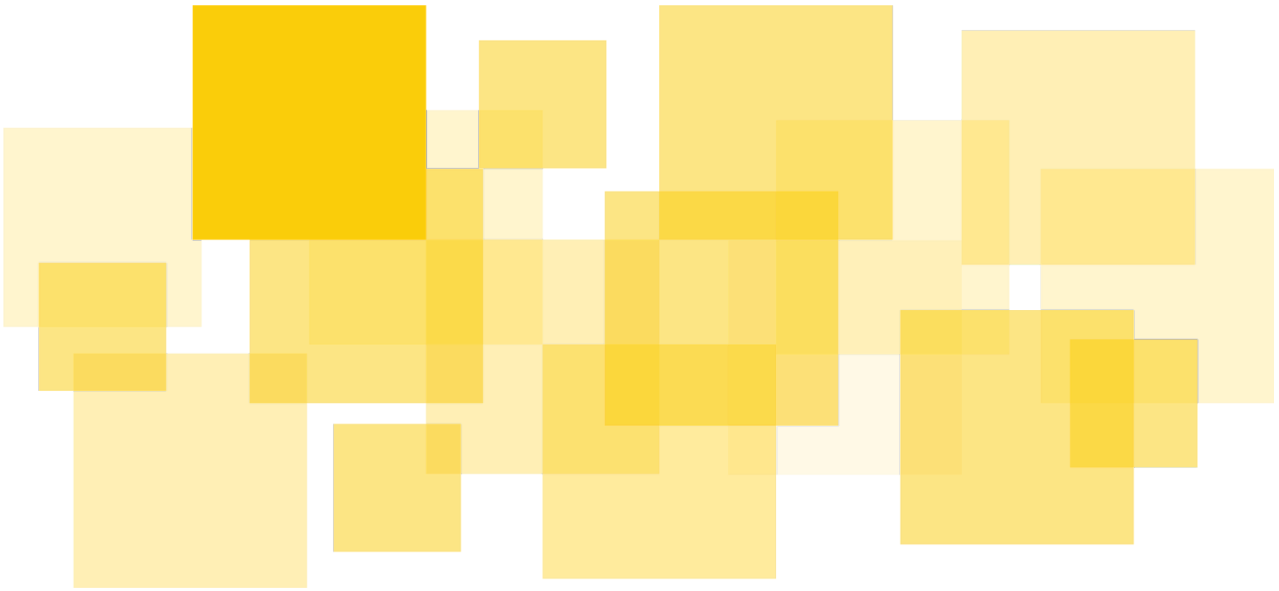
Security Audit Report

DeFindex - APY Stabilizer Stellar

Delivered: July 3, 2026



runtime verification

Table of Contents

Disclaimer

This report does not constitute legal or investment advice. You understand and agree that this report relates to new and emerging technologies and that there are significant risks inherent in using such technologies that cannot be completely protected against. While this report has been prepared based on data and information that has been provided by you or is otherwise publicly available, there are likely additional unknown risks which otherwise exist. This report is also not comprehensive in scope, excluding a number of components critical to the correct operation of this system. This report is for informational purposes only and is provided on an "as-is" basis and you acknowledge and agree that you are making use of this report and the information contained herein at your own risk. The preparers of this report make no representations or warranties of any kind, either express or implied, regarding the information in or the use of this report and shall not be liable to you or any third parties for any acts or omissions undertaken by you or any third parties based on the information contained herein.

Smart contracts are still a nascent software arena, and their deployment and public offering carries substantial risk.

Finally, the possibility of human error in the manual review process is very real, and we recommend seeking multiple independent opinions on any claims which impact a large quantity of funds.

Executive Summary

Paltalabs engaged Runtime Verification Inc. to perform a security audit of two DeFindex smart contracts, FeeProxy & BoostTreasury. The audit was conducted between June 16, 2026 and June 30, 2026. The objective was to assess the implementation's security and correctness, identify exploitable vulnerabilities, and provide recommendations to enhance the system's reliability.

DeFindex is a multi-asset yield aggregator on the Stellar network: partner vaults accept user deposits and route them into underlying yield strategies. The APY Stabilizer is an add-on that lets partners offer their users a stable, predictable APY instead of one that tracks volatile strategy yields. It comprises two independent Soroban contracts, FeeProxy and BoostTreasury, coordinated by an off-chain service.

The audit spanned two calendar weeks, with two Soroban auditors working in parallel. The work combined a high-level design review with a thorough manual review of the contracts' Soroban Rust source. We analyzed the architecture, the trust assumptions behind each role, and the security implications of the contracts' interactions with DeFindex vaults and SEP-41 tokens on Stellar. Emphasis was placed on identifying the core invariants and properties the system must uphold, and on checking the implementation against them across all state transitions.

Throughout the engagement, findings were shared with the DeFindex team in real time, which allowed for iterative discussion and clarification and accelerated the fix-and-verify cycle.

The audit resulted in findings ranging in severity from `[Medium]` to `[Informative]`. These findings have been organized into the following sections:

- **Findings of Severity**: considers findings that constitute threats to the protocol's health or to user and protocol funds;
- **Informative Findings**: considers findings that constitute general improvements or potential enhancements to the system's overall design and implementation.

Scope

The scope of this audit was limited to the code contained in the public GitHub repository provided by the DeFindex team, located at https://github.com/defindex-io/stellar-apy-stabilizer, at the fixed commit hash 6b2b2a43ed8e43972ecff3810bdd12cdc951fac9 on the main branch, which served as the initial revision under review. The elements within scope are divided into two core components, both implemented as Soroban smart contracts for the Stellar network:

`./contracts/fee-proxy/src/` : Contains the implementation of the **FeeProxy** contract, including its role-based access control system (`Admin` , `FeeManager` , and per-vault `Admin` roles), two-step admin rotation, vault registration and unregistration, fee bounds enforcement, fee management operations (lock, distribute, release), and passthrough functions for vault management operations. Approximately 583 lines of Rust across 4 source files, excluding tests.

`./contracts/boost-treasury/src/` : Contains the implementation of the **BoostTreasury** contract, including its role-based access control system (`Admin` and `Manager` roles), two-step admin rotation, boost campaign lifecycle management, token deposit and boost injection flows, campaign budget accounting, and orphan token recovery. Approximately 645 lines of Rust across 4 source files, excluding tests.

The DeFindex Vault source code, located at https://github.com/paltalabs/defindex/tree/main/apps/contracts/vault/src, was consulted as a reference to verify function signatures and access control for the cross-contract calls made by FeeProxy, but was not itself subject to a full security review. The APY stabilizer bot, off-chain tooling, deployment infrastructure, and any other components outside the two directories listed above are explicitly excluded from this engagement.

As findings are identified during the review, the client may submit remediation commits addressing reported issues. These commits will also be reviewed to verify that the identified issues are appropriately resolved prior to the final report.

Methodology

Runtime Verification followed the methodology below to evaluate the correctness and security of the DeFindex smart contracts FeeProxy & BoostTreasury within a two-week engagement, with two auditors working in parallel.

Design Review and Manual Code Review

Although manual code review cannot guarantee the discovery of all possible security vulnerabilities, as noted in our Disclaimer, we followed a structured and thorough approach to maximize the effectiveness of this audit engagement within the agreed timeframe.

The design review and manual code review phase spanned two calendar weeks, with the process being designed to identify and validate both high-level and low-level security concerns. During the two weeks, we conducted a high-level design review of the FeeProxy and BoostTreasury contracts that interact with the DeFindex Vault smart contract. We analyzed the architecture, key trust assumptions, and the security implications of interacting with various liquidity venues on the Stellar network. Emphasis was placed on identifying core invariants and properties that should be upheld by the protocol. We thoroughly reviewed the contracts' source code to detect any unexpected (and possibly exploitable) behaviors.

This approach enables us to systematically check consistency between the logic and the provided Soroban Rust implementation of the system.

Findings and Reporting

Findings in this report stem from a combination of:

- Manual inspection of the source code.
- Design-level reasoning about protocol behavior and system invariants.
- Analysis of abstractions and threat models against the invariants and properties.

Throughout the audit engagement, findings were reported to the DeFindex team in real-time, allowing for iterative discussion and clarification, which accelerated the fix-and-verify cycle.

Platform Features & Descriptions

DeFindex is a yield-aggregation protocol on Stellar that lets users deposit assets into tokenized vaults which route capital across a configurable set of yield-generating strategies. Depositors receive vault share tokens (dfTokens) representing proportional ownership, and each vault is governed by a set of on-chain roles (Manager, RebalanceManager, EmergencyManager, VaultFeeReceiver) that oversee strategy allocations, fees, and emergency actions.

The two contracts in scope for this audit, collectively referred to as the APY Stabilizer, extend that base system with a managed fee policy and an optional yield-boost campaign that a vault partner can opt into.

FeeProxy

The **FeeProxy** exists to solve a delegation problem: DeFindex needs to autonomously adjust vault fees on behalf of partners to maintain a target APY, but the vault's `Manager` role — the only role that can call `lock_fees` — also grants powers that should never be given to an automated bot (vault upgrades, role changes, fund rescue).

The proxy achieves this by becoming the vault's `Manager` itself, but internally splitting permissions into two tiers: a tier for the bot, which should only be able to call `lock_fees` and `distribute_fees`, and a full-passthrough tier for the partner. Partners trade direct Manager access for a mediated interface that preserves all their control while enabling DeFindex's automation.

The stated design goals are:

- Partners retain full effective control over their vault
- DeFindex can adjust fees autonomously, within partner-defined bounds
- A single deployed contract serves all partner vaults
- The contract is not upgradable — partners can trust its behavior is fixed

Architecture

The `FeeProxy` is a single contract managing an unbounded set of partner vaults. Each vault is registered with a `VaultConfig` entry stored in a per-vault persistent map:

```
VaultConfig {
    admin: Address,       // Partner's address — controls their vault through the proxy
    target_apy_bps: u32,  // Target APY in basis points - read by the bot off-chain
    min_fee_bps: u32,     // Minimum fee (usually 0)
    max_fee_bps: u32,     // Maximum fee the bot can set (safety cap)
}
```

There are three global actors:

Actor	Address stored	Scope
DeFindex admin	instance storage (`Admin`)	global — manages the proxy itself
Bot (`fee_manager`)	instance storage (`FeeManager`)	global — fee ops on every registered vault
Partner (`config.admin`)	per-vault persistent storage	per-vault — all operations on their own vault

When a partner registers, the proxy calls `vault.set_manager(proxy_address)` atomically in the same transaction. From that point on, all Manager-level calls to the vault must go through the proxy.

Trust Model

Actors and their roles

Actor	Permissions
DeFindex admin	- Rotate the global `fee_manager` address (`set_fee_manager`) - Transfer `admin` ownership via two-step propose/accept
Bot (`fee_manager`)	- `lock_fees` on any registered vault within that vault's `[min_fee_bps, max_fee_bps]` - `distribute_fees` on any registered vault
Partner (`config.admin`)	- `lock_fees` within bounds - `distribute_fees` - Set `target_apy_bps` and fee bounds - `release_fees` - Upgrade vault - Change all vault roles - Rescue, pause/unpause strategies - Unregister vault (reclaims Manager role in one transaction)

Trust assumptions

The design rests on three trust assumptions that are worth making explicit:

1. **The bot computes the correct `fee_bps`**
   The proxy validates that the submitted `fee_bps` is within the range [`min_fee_bps` , `max_fee_bps`], but does not verify that it corresponds to `target_apy_bps`. The target is stored on-chain for transparency, but its enforcement is entirely off-chain. A bot that ignores the target and always submits `max_fee_bps` would pass all on-chain checks. The partner's only protection is the bounds they set and their ability to override the bot.

2. **The indexer data the bot reads is accurate**
   The bot computes gross APY from the indexer's historical PPS snapshots. If the indexer is compromised, stale, or returns incorrect data, the bot will set the wrong fee. The bounds check is again the only on-chain defense.
3. **The partner's `config.admin` key is secure**
   The partner admin has effectively the same powers over their vault as they had before (all Manager operations, plus the ability to override bot fees). A compromised partner admin key poses the same risk as before delegating the manager to the proxy.

Properties to Verify

P1. Delegating Manager to FeeProxy does not open any access control vulnerability on the vault
Delegating the Manager role to the FeeProxy should not expose the vault to operations it would not face with a self-managed Manager. We should verify that:

- No FeeProxy role (admin or fee_manager) or any random address can reach a vault operation that it is not entitled to.
- Cross-contract calls made by the proxy should not be exploitable by a malicious actor or a malicious vault implementation to re-enter the proxy in an inconsistent state.

P2. Role boundaries and fee bounds enforcement
Each entrypoint's allowed-caller set should match the intended permission model, and any fee set through the proxy should always fall within the vault's configured bounds. We should verify that:

- Only `fee_manager` or `config.admin` can call `lock_fees` and `distribute_fees`; all other callers are rejected.
- Only `config.admin` can call config-changing and passthrough entrypoints; all other callers are rejected (including `fee_manager` and global admin).

- A fee submitted via `lock_fees` is always within [`min_fee_bps` , `max_fee_bps`] for the target vault.
- `validate_fee_bounds` is called on every path that writes fee bounds, and correctly rejects `min > max` and `max > 10_000` .

P3. Registration and unregistration integrity

The proxy's stored config and the vault's on-chain Manager role should always be consistent. We should verify that:

- After a successful `register_vault` , the vault's Manager is the proxy and a `VaultConfig` entry exists for that vault.
- After a successful `unregister_vault` or `set_vault_manager` (to a non-proxy address), the vault's Manager is no longer the proxy and no `VaultConfig` entry remains.
- There is no execution path that leaves the proxy holding the vault's Manager role without a corresponding config entry, or that leaves a config entry without the proxy being Manager.
- `register_vault` cannot be called by an address that is not the vault's current Manager.

P4. Vault isolation

A partner's admin should have no influence over any other vault registered with the proxy. We should verify that:

- Each vault's configuration is loaded and authenticated independently — a caller authorized for vault A cannot affect vault B.
- There is no path through which one vault's config read or write can affect another vault's config.
- The global `fee_manager` or global admin cannot modify any vault's VaultConfig (target APY, fee bounds, admin).

P5. Admin transfer safety

The two-step propose/accept pattern for admin rotation should be safe against lockout

and unauthorized takeover. We should verify that:

- Only the current admin can call `propose_admin`.
- Only the exact address stored as `pending_admin` can call `accept_admin`.
- A proposal can be overwritten or canceled by the current admin before acceptance.
- There is no state in which both `admin` and `pending_admin` are unset or unresponsive, locking the proxy permanently.

BoostTreasury

The **BoostTreasury** exists to solve a subsidy problem. When a vault's realized APY drops below the partner's target APY, the difference can be made up by injecting funds into the vault via a regular token transfer to the vault contract. Those funds have to be stored somewhere in-between APY boosts and operated by a bot that initiates token transfers.

The treasury does this by acting as a per-vault escrow that separates funding from disbursement. Anyone can pre-fund a campaign's budget. The bot, which holds the `manager` role, can stream that budget into the one vault the campaign is registered for, but never more than has been deposited and never to any other address. The DeFindex admin keeps custody for refunds.

The stated design goals are:

- Anyone can pre-fund a per-vault boost budget, and funds are tracked per campaign
- DeFindex can disburse boosts autonomously, bounded by the deposited budget and directed only at the registered vault
- A single deployed contract serves all partner vaults
- The DeFindex admin keeps custody for refunds, reallocation, and recovery of tokens sent in by mistake
- The contract is not upgradable, so depositors can trust its behavior is fixed

Architecture

The `BoostTreasury` is a single contract managing a set of per-vault campaigns. Each vault is registered with a `Campaign` entry stored as a ledger entry keyed by the vault address, and the set of registered vaults is mirrored in a persistent `CampaignList` so tracked balances can be enumerated per asset:

```
Campaign {
    active: bool,           // deposits and boosts require this, transfer and rescue do
not
```

```
    asset: Address,          // the vault's single asset, captured at registration and
immutable
    total_deposited: i128,   // cumulative amount deposited into the campaign
    total_boosted: i128,     // cumulative amount streamed to the vault
    total_withdrawn: i128,   // cumulative amount transferred out by the admin
    last_boosted_at: u64,    // ledger timestamp of the last boost, off-chain monitoring
only
}
// available() = total_deposited - total_boosted - total_withdrawn
```

There are two global actors and one permissionless participant:

Actor	Address stored	Scope
DeFindex admin	instance storage (`Admin`)	global, manages campaigns and holds custody of all budgets
Bot (`manager`)	instance storage (`Manager`)	global, `boost` on every active campaign
Depositor	not stored	permissionless, funds any active campaign

A campaign is created by the admin. Although DeFindex vaults may hold multiple assets, the treasury supports only single-asset vaults. The contract reads `vault.get_assets()` at registration and rejects any vault that reports more than one asset. From then on the campaign uses that single stored asset for every token movement, whether a deposit coming in, a boost going to the vault, or an admin transfer going out. A `boost` is a plain token transfer into the vault, which raises the vault's idle funds and therefore its price-per-share for all shareholders.

Trust model

Actors and their roles

Actor	Permissions
DeFindex admin	- Register, activate or deactivate, and unregister campaigns - `transfer` a campaign's available budget to any address (refunds, reallocation) - `rescue_orphan` to sweep tokens that arrived outside `deposit` - Rotate the global `manager` address (`set_manager`) - Transfer `admin` ownership through two-step propose/accept
Bot (`manager`)	- `boost` an active campaign, up to its available budget and only to the registered vault
Depositor (anyone)	- `deposit` into any active campaign

Trust assumptions

The design rests on four trust assumptions worth making explicit:

1. **The bot boosts the right amount at the right time**
   The treasury enforces that a boost cannot exceed the campaign's available budget, that the campaign is active, and that funds go only to the registered vault. It does not check that the boost corresponds to any APY shortfall. A manager that drains the full available budget in a single boost would pass every on-chain check. The depositor's only protection is the accounting bound, the fixed destination, and the admin's ability to deactivate the campaign.
2. **The DeFindex admin is a trusted custodian of campaign budgets**
   By design, the admin can call `transfer` to send any campaign's available budget to any address, and can deactivate a campaign at any time. Depositors must trust the admin to use this only for legitimate refunds and reallocation. This custody is the central trust of the escrow model, and it is intentional. A compromised admin key therefore carries the same weight here as it does for any custodial treasury, and the admin is expected to be a multisig or cold key.
3. **The campaign asset is a well-behaved SEP-41 token**
   All accounting assumes `token.transfer` moves exactly `amount` units and `token.balance` reports honestly. A non-standard asset, such as a fee-on-transfer or rebasing token or one with a malicious implementation, would let the contract's tracked accounting drift from its real balance and break both the available-budget bound and the orphan-rescue computation. Because the asset is captured from the vault's own `get_assets()` report at registration, this reduces to trusting the registered vault's asset.
4. **Registered campaigns point to genuine DeFindex vaults**
   `register_campaign` is admin-only, and the only thing it verifies about the target is that it answers `get_assets()` with a single asset. It cannot tell a real, properly deployed DeFindex vault from any other contract that exposes a matching

`get_assets()`, so nothing on-chain stops a campaign from being registered against a wrong or hostile address.

Properties to verify

P1. Treasury solvency and accounting integrity
The treasury should never disburse more than has been deposited, and its tracked budget should never exceed its real token balance. We should verify that:

- For every campaign, `available() == total_deposited - total_boosted - total_withdrawn` and is always ≥ 0, with no reachable path that overflows or underflows it.
- A `deposit` increases `total_deposited` by exactly the amount transferred in, while a `boost` or `transfer` increases `total_boosted` or `total_withdrawn`, respectively, by exactly the amount sent out.
- Neither `boost` nor `transfer` can move more than the campaign's `available()`.
- For any asset, the contract's real token balance is always ≥ the sum of `available()` across every campaign using that asset, and `rescue_orphan` can move only the surplus above that sum.
- The only cross-contract calls are the SEP-41 `transfer` and `balance` and the vault's `get_assets`, and none of them leaves the accounting inconsistent. Soroban disallows reentrancy, so a token or vault the treasury calls cannot re-enter it mid-call, and the order of token transfer and accounting update in `deposit`, `boost`, and `transfer` carries no reentrancy risk.

P2. Role boundaries and input validation
Each entrypoint's allowed-caller set should match the intended permission model. We should verify that:

- Only `manager` can call `boost`, and all other callers are rejected.
- `register_campaign`, `update_campaign`, `unregister_campaign`, `transfer`, `rescue_orphan`, and `set_manager` are restricted to `admin`, and every other caller, including `manager` and depositors, is rejected.

- `deposit` authenticates `caller` and pulls funds from exactly that authenticated address.
- Every value-moving entrypoint rejects non-positive amounts.
- `deposit` and `boost` require the campaign to be active. `transfer` and `rescue_orphan` deliberately do not, which gives the admin an escape hatch from a deactivated campaign.

P3. Campaign registration and asset integrity

A campaign's stored asset and lifecycle state should always be consistent, and immutable where intended. We should verify that:

- `register_campaign` rejects multi-asset vaults.
- A vault cannot be registered twice.
- Once set, a campaign's `asset` never changes, and every token movement for that campaign uses it.
- `unregister_campaign` succeeds only when `available() == 0`, and removes both the `Campaign` entry and its `CampaignList` membership.
- The `Campaign` map and the `CampaignList` stay consistent: every listed vault has a campaign and every campaign is listed, with no path that desynchronizes them.

P4. Campaign isolation

An operation on one campaign should have no effect on any other. We should verify that:

- Each campaign's accounting is loaded and updated independently, so a deposit, boost, or transfer on one vault cannot change another vault's totals.
- `available()` for a campaign is derived only from that campaign's own totals.
- `rescue_orphan` attributes tracked balances to the correct asset across campaigns, and can never sweep funds belonging to a campaign that uses the swept token.

P5. Admin transfer safety

The two-step propose/accept pattern for admin rotation should be safe against lockout and unauthorized takeover. We should verify that:

- Only the current admin can call `propose_admin`.

- Only the exact address stored as `pending_admin` can call `accept_admin`.
- A proposal can be overwritten or canceled by the current admin before acceptance.
- There is no reachable state in which both `admin` and `pending_admin` are unset, which would lock the treasury permanently.

Findings

This section contains all issues identified during the audit that could lead to unintended behavior, security vulnerabilities, or failure to enforce the protocol's intended logic. Each issue is documented with a description, potential impact, and recommended remediation steps.

A01 - BoostTreasury - `rescue_orphan` : unbounded ledger entry reads can make the function uncallable

Severity: Low | Difficulty: Low | Recommended Action: Fix Design | Addressed by client

The `rescue_orphan` function in BoostTreasury iterates over all registered vaults in `CampaignList` and reads each vault's `Campaign` entry individually from persistent storage to compute the total tracked balance for a given token:

```
let list = storage::get_campaign_list(&env);
let mut tracked: i128 = 0;
for vault in list.iter() {
    if let Some(campaign) = storage::get_campaign(&env, &vault) {
        if campaign.asset == token {
            tracked += campaign.available();
        }
    }
}
```

Each `storage::get_campaign` call reads a distinct persistent ledger entry. According to the Stellar Lab Network Limits page (consulted June 2026), Soroban currently enforces a hard limit of 200 disk read entries per transaction on mainnet. Accounting for the fixed reads (`Admin` , `CampaignList` , and the token balance call), `rescue_orphan` becomes uncallable once approximately 197 vaults are registered — the transaction would fail, hitting the read entry limit.

Impact:

If the number of registered campaigns reaches the threshold, `rescue_orphan` becomes temporarily uncallable. The `admin` can restore access by unregistering enough campaigns to bring the total below the limit. However, `unregister_campaign` requires

`campaign.available() == 0`, meaning campaigns with remaining budget cannot be unregistered, which limits the ability of `admin` to quickly recover access in practice.

During the period where `rescue_orphan` is uncallable, tokens that arrive outside of `deposit()` — direct transfers, dust, mistaken sends — cannot be recovered.

Recommendation:

Two mitigations are possible, and can be applied independently or together:

- Enforce a maximum campaign limit — reject `register_campaign` if the number of registered campaigns already reaches a safe threshold, preventing the issue from ever being triggered.
- Replace the per-call aggregation with a per-token tracked balance counter maintained incrementally. Add a new storage key:

```
TrackedBalance(Address),  // Address = token address
```

Update it on every `deposit`, `boost`, and `transfer`:

- `deposit` → increment `TrackedBalance(asset)` by amount
- `boost` → decrement `TrackedBalance(asset)` by amount
- `transfer` → decrement `TrackedBalance(asset)` by amount

Then `rescue_orphan` reduces to two ledger reads regardless of the number of registered vaults:

```
let tracked = storage::get_tracked_balance(&env, &token);
let orphan = balance - tracked;
```

Status:

Addressed in commit defindex-io/stellar-apy-stabilizer @8fcacb4 with the second suggestion: replaced the O(n) `CampaignList` scan in `rescue_orphan` with an

incrementally maintained per-token total (`Tracked(token)`), updated on `deposit` / `boost` / `transfer` . The `rescue_orphan` function is now O(1) and the `CampaignList` vector is removed.

A02 - BoostTreasury - Design: Sandwich Attacks and MEV on Boosts

Severity: Medium | Difficulty: Medium | Recommended Action: Fix Design | Acknowledged by client

The smart contract BoostTreasury sends funds to a DeFindex vault by means of a regular token transfer. This increases the TVL of that vault and increases the owned funds by all vault shareholders. This is entirely intended, as the BoostTreasury contract is used to increase the APY for all vault shareholders.

Since the boost is a periodic transfer of funds to the DeFindex vault, a malicious actor can also profit from BoostTreasury boosts without being a long-term stakeholder of a vault. Instead, by sandwiching deposit and withdraw transactions around a BoostTreasury boost transaction, the malicious actor, as a temporary shareholder, collects part of the boost.

While sandwich attacks are particularly challenging on Stellar, they still cannot be fully ignored. Additionally, the boosts might happen at predictable points in time, allowing malicious actors to sandwich attack without having seen the transactions, and withdrawing as soon as they do.

Impact:

The BoostTreasury design is potentially susceptible to Maximal Extractable Value (MEV) attacks.

Recommendation:

We recommend having multiple more frequent boosts rather than less frequent, larger boosts, since this decreases the MEV per boost and reduces the incentive for malicious actors to MEV attack the contract. Further, making the boost timing slightly less

predictable can also make sandwich attacks that rely on predicting the exact timing of boost transactions harder to execute.

We also want to emphasize that definitive conclusions about the profitability of MEV attacks for a given periodic boost configuration should be based on empirical analysis.

Further, we want to highlight that sandwich attacks should be fully considered when designing and implementing the BoostTreasury boost bot that creates the boost transactions. It should be ensured that a malicious actor cannot, or only partially, increase the boost amount by temporarily increasing the TVL of a DeFindex vault.

Status:

Acknowledged by the client. They will take this into consideration in the boost bot design and plan on running the boost at least on an hourly basis, which reduces the extractable funds compared to longer periods. They plan on properly documenting the mitigations implemented in the bot and are considering additional mitigations.

Informative Findings

This section includes observations that are not directly exploitable, but highlight areas for improvement in code clarity, maintainability, or best practices. While not critical, addressing these can strengthen the system's overall robustness.

B01 - FeeProxy - `register_vault` : redundant `admin` parameter can be removed

Severity: Informative | Recommended Action: Fix Code | Addressed by client

The `register_vault` function in `FeeProxy` takes an explicit `admin` argument and then immediately checks that it matches `config.admin` :

```
pub fn register_vault(env: Env, admin: Address, vault: Address, config: VaultConfig) {
    admin.require_auth();
    if admin != config.admin {
        panic_with_error!(&env, ContractError::AdminMismatch);
    }
    ...
}
```

Since `config.admin` is always required to equal `admin` , the separate parameter is redundant. The function could be simplified to:

```
pub fn register_vault(env: Env, vault: Address, config: VaultConfig) {
    config.admin.require_auth();
    ...
}
```

This eliminates the `AdminMismatch` error case entirely since there is nothing to mismatch — the signer is always `config.admin` by construction.

Impact:

No security impact. The current implementation correctly enforces the invariant via the `AdminMismatch` check. This is a code simplification only.

Recommendation:

Remove the `admin` parameter and call `config.admin.require_auth()` directly. Remove the `AdminMismatch` check and the `ContractError::AdminMismatch` variant as they become unreachable.

Status:

Addressed in commit defindex-io/stellar-apy-stabilizer @2e25388

B02 - FeeProxy - `release_fees` : missing event emission

Severity: Informative | Recommended Action: Fix Code | Addressed by client

The FeeProxy function `release_fees` is the only fee management function that does not emit an event after execution. `lock_fees` and `distribute_fees` both emit events, but `release_fees` silently completes:

```
pub fn release_fees(env: Env, vault: Address, strategy: Address, amount: i128) {
    extend_instance_ttl(&env);
    require_vault_admin(&env, &vault);

    if amount <= 0 {
        panic_with_error!(&env, ContractError::InvalidAmount);
    }

    call_vault(&env, &vault, "release_fees", vec![&env, strategy.into_val(&env),
amount.into_val(&env)]);
    // no event emitted
}
```

This is particularly relevant since `release_fees` involves an actual asset transfer to a strategy — arguably the most significant of the three fee operations — making observability more important, not less.

Impact:

No security impact. However, the missing event reduces observability for off-chain indexers, dashboards, and monitoring tools that track fee flows. Inconsistency with the other fee functions also makes the API harder to reason about.

Recommendation:

Add a `FeesReleased` event to `events.rs` and emit it at the end of `release_fees`:

```
#[contractevent]
pub struct FeesReleased {
    #[topic]
    pub vault: Address,
    #[topic]
    pub strategy: Address,
    pub amount: i128,
}

events::FeesReleased { vault, strategy, amount }.publish(&env);
```

Status:

Addressed in commit defindex-io/stellar-apy-stabilizer @072ff80

B03 - FeeProxy - `require_fee_manager_or_vault_admin`: duplicated code can be simplified

Severity: Informative | Recommended Action: Fix Code | Addressed by client

The function checks two conditions in separate `if` blocks that share identical bodies, leading to unnecessary code duplication:

```
fn require_fee_manager_or_vault_admin(env: &Env, caller: &Address, vault: &Address) ->
VaultConfig {
    let config = storage::get_vault_config(env, vault)
        .unwrap_or_else(|| panic_with_error!(env, ContractError::VaultNotRegistered));

    let fee_manager = storage::get_fee_manager(env);
    if *caller == fee_manager {
        caller.require_auth();
        return config;
    }
    if *caller == config.admin {
        caller.require_auth();
        return config;
    }
    panic_with_error!(env, ContractError::Unauthorized);
}
```

Both branches execute the same code, so they can be merged into a single condition:

```
fn require_fee_manager_or_vault_admin(env: &Env, caller: &Address, vault: &Address) ->
VaultConfig {
    let config = storage::get_vault_config(env, vault)
        .unwrap_or_else(|| panic_with_error!(env, ContractError::VaultNotRegistered));

    let fee_manager = storage::get_fee_manager(env);
    if *caller == fee_manager || *caller == config.admin {
```

```
        caller.require_auth();
        return config;
    }
    panic_with_error!(env, ContractError::Unauthorized);
}
```

Impact:

No security impact. The logic is equivalent and the current implementation correctly enforces access control. This is a code simplification only.

Recommendation:

Merge the two if blocks into a single condition as shown above.

Status:

Addressed in commit defindex-io/stellar-apy-stabilizer @e87f465

B04 - FeeProxy & BoostTreasury - Usage of Long Symbols in Ledger Entry Keys

Severity: Informative | Recommended Action: Fix Code | Addressed by client

Both contracts, `FeeProxy` and `BoostTreasury`, key their ledger entries with a `#[contracttype]` enum:

```
#[contracttype]
#[derive(Clone)]
pub enum DataKey {
    Admin,
    PendingAdmin,
    FeeManager,
    VaultConfig(Address),
}
```

When such an enum is converted to a Soroban `Val`, each variant becomes a vector of one or two elements: the first holds a symbol with the variant name, and the second, if present, holds the variant's associated value. A `#[contracttype]` enum variant may carry at most one such value. The variant name therefore appears in the ledger key as a symbol on every storage access.

Symbol length decides how that symbol is stored. A symbol may hold up to 32 characters, but only symbols of 9 characters or fewer use the compact `SymbolSmall` encoding, which packs the string into a 64-bit value instead of allocating a host object, see the `soroban_sdk::Symbol` documentation. Several `DataKey` variants exceed that limit, such as `PendingAdmin` (12 characters) and `FeeManager` (10 characters).

Impact:

This is a gas optimization only and has no security impact.

Recommendation:

Rename every `DataKey` variant longer than 9 characters so its symbol fits the `SymbolSmall` encoding, keeping within the `[a-zA-Z0-9_]` character set. This applies to the `DataKey` enums in both `FeeProxy` and `BoostTreasury`. The example below shows a fix for `FeeProxy` only:

```
#[contracttype]
#[derive(Clone)]
pub enum DataKey {
    Admin,
    PendAdmin,          // PendingAdmin
    FeeMang,            // FeeManager
    VaultConf(Address), // VaultConfig
}
```

Status:

This finding has been addressed in commit defindex-io/stellar-apy-stabilizer @921a099

B05 - `rebalance` : no passthrough in FeeProxy prevents Manager from rebalancing

Severity: Informative | Recommended Action: Fix Code | Acknowledged by client

The vault's `rebalance` function accepts calls from either the `Manager` or the `RebalanceManager`. When a vault is registered with FeeProxy, the proxy takes over the `Manager` role — meaning any `Manager`-authorized call must be routed through FeeProxy. However, FeeProxy does not expose a passthrough for `rebalance`.

As a result, once a vault is registered, the Manager's ability to rebalance is silently lost. The only remaining path to trigger a `rebalance` is through the `RebalanceManager` role, which operates independently of FeeProxy. This creates an asymmetry: a vault registered with FeeProxy has fewer operational capabilities than one that is not.

Impact:

The vault admin cannot trigger a `rebalance` via the `Manager` role while the vault is under FeeProxy's control. Depending on the deployment setup, this may leave the protocol without a rebalancing path if the `RebalanceManager` is not configured or available.

Recommendation:

Add a passthrough function following the same pattern as the other passthrough functions:

```
pub fn rebalance_vault(env: Env, vault: Address, instructions:
soroban_sdk::Vec<soroban_sdk::Val>) {
    extend_instance_ttl(&env);
    require_vault_admin(&env, &vault);
    let proxy = env.current_contract_address();
```

```
    call_vault(&env, &vault, "rebalance", vec![&env, proxy.into_val(&env),
instructions.into_val(&env)]);
}
```

Status:

Acknowledged, the recommendation will not be implemented. The client provided the following rationale:

- Rebalancing is not lost. FeeProxy assumes only the `Manager` role upon vault registration; the `RebalanceManager` role remains with the partner. Since the vault's rebalance function accepts either `Manager` or `RebalanceManager`, the partner retains a fully functional rebalance path through the role they still hold.
- Recoverable in edge cases. FeeProxy exposes `set_vault_rebalance_manager` (vault-admin-gated), allowing the partner to re-point the `RebalanceManager` to any address they control if needed — without requiring a Manager-path passthrough.
- Matches the documented design. Rebalancing is intentionally excluded from the minimal Manager-proxy surface and is planned for a dedicated `RebalanceManager` contract in Phase 3 of the protocol roadmap (Appendix A). Adding a Manager-path passthrough would duplicate an existing capability while unnecessarily widening the proxy's trust surface.

The client will instead make the `RebalanceManager` path explicit in partner onboarding documentation.

The auditors consider this rationale acceptable. The rebalance capability is not lost — it remains accessible through the `RebalanceManager` role — and the recovery path via `set_vault_rebalance_manager` addresses the edge case.

B06 - FeeProxy & BoostTreasury - Gas optimization: Avoid Constructing Unnecessary Clones

Severity: Informative | Recommended Action: Fix Code | Addressed by client

The BoostTreasury smart contract constructs a new `Campaign` object instead of mutating the existing object in place, see function `transfer`:

```
let new_total_withdrawn = campaign
    .total_withdrawn
    .checked_add(amount)
    .unwrap_or_else(|| panic_with_error!(&env, ContractError::Overflow));
let updated = Campaign {
    total_withdrawn: new_total_withdrawn,
    ..campaign
};
storage::set_campaign(&env, &vault, &updated);
```

Similarly, the function `set_vault_manager` in the FeeProxy smart contract uses `new_manager.clone().into_val(&env)`, which can be simplified to `new_manager.into_val(&env)` since `into_val` borrows `&self`.

See code:

```
pub fn set_vault_manager(env: Env, vault: Address, new_manager: Address) {
    extend_instance_ttl(&env);
    let config = require_vault_admin(&env, &vault);
    // the `.clone()` can be dropped since `into_val` borrows `&self`
    call_vault(&env, &vault, "set_manager", vec![&env,
new_manager.clone().into_val(&env)]);
    if new_manager != env.current_contract_address() {
    [...]
```

Impact:

These are gas optimizations only and have no security impact.

Recommendation:

Make `updated` a mutable variable in the BoostTreasury function `transfer` and mutate it in place instead of constructing a new object.

In FeeProxy, drop the unnecessary `.clone()` in the function `set_vault_manager` as outlined above.

Status:

This finding has been addressed in commits defindex-io/stellar-apy-stabilizer @8fcacb4 and defindex-io/stellar-apy-stabilizer @21652f7

B07 - BoostTreasury - Gas Optimization: Mutate the Host Vector with a Single Host Function Call instead of a Loop

Severity: Informative | Recommended Action: Fix Code | Addressed by client

In the function `unregister_campaign` in the smart contract BoostTreasury, an element is removed from a soroban host vector by elementwise constructing a new soroban host vector:

```
let list = storage::get_campaign_list(&env);
let mut new_list = Vec::new(&env);
for v in list.iter() {
    if v != vault {
        new_list.push_back(v);
    }
}
storage::set_campaign_list(&env, &new_list);
```

This code snippet requires 1 host function call to create an empty vector, n host function calls to get the elements via the iterator, and $n - 1$ host function calls to add all previous elements elementwise excluding the desired element to remove. Each interim host vector is a new immutable host object stored alongside other host objects and in total this requires $2n$ host function calls, excluding the `storage::get` and `storage::set` functions in the code snippet.

Since host vectors are immutable, this can safely be optimized by using the soroban sdk functions `Vec::first_index_of`, see here, and `Vec::remove`, see here. These make use of the host functions `vec_first_index_of` and `vec_del`, respectively. Under the assumption that campaigns contains no duplicate vault entries, the number of host function calls can be reduced from $2n$ to 2. This also reduces the total number of host objects allocated.

Impact:

This is a gas optimization only and has no security impact.

Recommendation:

Replace the loop that constructs a new vector elementwise with calls to `Vec::first_index_of` and `Vec::remove`. The example below assumes that campaigns contains no duplicate vault entries. The code can be further extended to also remove duplicate entries, if desired:

```
let mut list = storage::get_campaign_list(&env);
if let Some(i) = list.first_index_of(&vault) {
    list.remove(i);
    storage::set_campaign_list(&env, &list);
}
```

Status:

This finding has been subsumed by the fix for A01 that was addressed in commit defindex-io/stellar-apy-stabilizer @8fcacb4

B08 - BoostTreasury - Reallocating Funds Requires Interim Wallet or Smart Contract

Severity: Informative | Recommended Action: Document Prominently | Addressed by client

In BoostTreasury, the smart contract function `transfer` allows the admin to transfer tokens from any registered vault to an address given by the admin.

This function is intended to be used by the admin to refund unspent budget to depositors and for reallocating funds between vaults. Specifically, the comment above the function states:

```
/// Sends `amount` tokens from a campaign's tracked budget to `to`. Admin
/// only. Used for refunds (returning unspent budget to depositors) or for
/// reallocating between vaults.
///
/// ⚠️ This function grants the admin role full power to drain any
/// campaign's `available()` budget to any address. Depositors should be
/// aware that admin can redirect their funds — this is by design, the
/// admin is the trusted operator of the treasury. For tokens that arrived
/// OUTSIDE `deposit()` (direct transfers, dust, mistaken sends), use
/// `rescue_orphan` instead, which is bounded by the campaign-tracked totals
/// and cannot accidentally move funds attributed to a campaign.
pub fn transfer(env: Env, vault: Address, amount: i128, to: Address) {
    [...]
```

However, the function only allows reallocation of funds between vaults by depositing funds in an interim wallet or smart contract and then redepositing.

Impact:

This finding has no impact on security and correctness. Though the recommendation helps avoid the need for interim wallets or smart contracts when the admin wishes to

reallocate funds from one vault to another.

Recommendation:

We recommend adjusting the comment to either not state that the function is intended to be used for reallocating funds, or implementing a dedicated function that allows the admin to directly reallocate funds from one vault to another.

Status:

This finding has been addressed in commit defindex-io/stellar-apy-stabilizer @8fcacb4 by adding a new endpoint `reallocate`

Additional Feedback

Prior to the beginning the review, the client requested us to investigate and provide additional feedback on the following areas of concern:

Vault attack surface from FeeProxy

Does delegating Manager to FeeProxy expose the vault to anything it wouldn't face with a self-managed Manager? Can any FeeProxy role (admin, fee_manager, vault config.admin) reach vault operations they shouldn't?

Feedback

Delegating the `Manager` role to `FeeProxy` does not expose the vault to any operations it wouldn't face under a self-managed `Manager`. Each FeeProxy role is correctly scoped:

- Global `admin` — can only manage global contract state (`fee_manager` rotation, `admin` transfer). Cannot reach any vault operation.
- `fee_manager` — can only call `lock_fees` and `distribute_fees`, bounded by `[min_fee_bps, max_fee_bps]`. Cannot reach any vault management operation.
- `VaultConfig.admin` — can reach all vault management operations through FeeProxy's passthrough functions for its own vault only. Storage is scoped per vault address, and every passthrough function validates the caller against the specific vault's config — a partner cannot reach operations on any other registered vault. The capabilities exposed match exactly what a self-managed `Manager` would have.

In terms of compromise impact: a compromised global `admin` could set a malicious `fee_manager`; a compromised `fee_manager` could set fees to any value within `[min_fee_bps, max_fee_bps]`; a compromised `config.admin` exposes its vault to the same risks it would face under a self-managed Manager.

FeeProxy role boundaries and fee bounds

Confirm each entrypoint's allowed-caller set is correct, and that any fee actually set by FeeProxy always stays within the vault's configured [min_fee_bps, max_fee_bps].

Feedback

The allowed-caller set for each entrypoint and the fee bounds enforcement are fully covered in the Trust Model of the FeeProxy. To summarize: each role is correctly scoped, no role can reach vault operations beyond their intended boundaries, and l`ock_fees(Some(bps))` always validates `min_fee_bps ≤ bps ≤ max_fee_bps` before forwarding to the vault.

BoostTreasury fund safety

The accounting invariant available = total_deposited - total_boosted - total_withdrawn must hold under every sequence. Funds should never be permanently locked or double-spent, and admin transfer (the escape hatch) shouldn't be abusable.

Feedback

The available-funds invariant is upheld by all endpoints in the BoostTreasury smart contract, given proper deployment. The endpoint `rescue_orphan` allows the admin to recover unaccounted funds. The admin role cannot be abused to violate the available funds invariant.

Admin compromise blast radius

For each privileged role on both contracts, what's the worst-case outcome if the key is compromised, and what mitigations would you recommend?

Feedback

- Admin: The Admin role can transfer funds deposited to campaigns outside of the vault and take absolute custody. The role can also reallocate funds in between campaigns. Funds that are not allocated to any campaign, transferred to the BoostTreasury by means of a regular transfer, can be transferred and taken custody of by the admin. Campaigns can also be unregistered and re-registered with a

different configuration. In case of a compromise, all funds of a BoostTreasury can be fully taken custody of in the worst case.

- Pending admin: The pending admin role can be used to get custody of the admin. Therefore, the worst-case outcome in case of compromise is the same as admin. The pending admin role can be overridden by the admin.
- Manager: The manager can call the `boost` endpoint that can transfer funds from any campaign to the campaign's designated DeFindex vault. In case of a compromise, the role can be used to fully drain all campaigns' funds, which are then transferred to the respective registered DeFindex vaults. By means of a sandwich attack, as described in A02 - BoostTreasury - Design: Sandwich Attacks and MEV on Boosts , a malicious actor can take custody of some of the transferred funds, depending on respective vault TVL and funds available to the malicious actor. The rest of the transferred funds would be distributed to the other respective shareholders of DeFindex vaults.

We recommend the utilization of a multi-sig wallet for the admin role as well as for pending admin. For manager, we recommend considering additional protocol-level safeguards such as rate limits combined with off-chain detection of anomalous boosts.